Ders 23 Çalışma Özeti

Ders 23 Çalışma Özeti

Genel Konular

- En küçük kapsayan ağaç (minimum spanning tree, MST) problemi: köy ve muhtar örneği.
- Jenerik MST algoritması: güvenli kenar (safe edge), kesim (cut) ve hafif kenar (light edge).
- Kruskal algoritması: ayrık kümelerle (disjoint set) çalışma, kitaptaki 9 düğümlü örnek, karmaşıklık.
- Prim algoritması: öncelikli kuyrukla (priority queue) tek bir ağacı büyütme, aynı örnek, karmaşıklık.
- Tek kaynaklı en kısa yol (single source shortest path) problemi ve kullanım alanları.
- Dijkstra algoritması: gevşetme (relax) işlemi, üç örnek (iki yönlü graf, bir yönsüz graf).
- Ödevdeki girdi okuma şekli hakkında açıklama.

Hocanın Özellikle Vurguladığı Kısımlar

- MST'de V düğüm $V-1$ kenarla bağlanır. Ağaçta döngü (cycle) olmamalı.
- Bir graf için birden fazla MST olabilir. Toplam ağırlıkları aynıdır. Eşit ağırlıklı kenarlarda seçim sırası değişirse farklı bir MST çıkar.
- Güvenli kenar: iki farklı bileşeni en ucuz şekilde bağlayan kenar. Aynı bileşendeki iki düğümü bağlayan kenar güvenli değildir, "boşuna masraftır".
- Kruskal ile Prim'in çalışma mantığı terstir:
  - Kruskal V ayrı bileşenle başlar, hafif kenarlarla bileşenleri birleştirir.
  - Prim her an tek bir ağaçla çalışır, bu ağacı en hafif kenarla büyütür.
- Karmaşıklık açısından ikisi arasında "aman aman bir kazanç yok": ikisi de $O(E \log V)$.
- Dijkstra yönlü ve yönsüz graflarda çalışır. Kenar ağırlıkları negatif olmamalı. Negatif ağırlıkta Bellman-Ford kullanılır.
- Dijkstra'nın Prim'den tek temel farkı: anahtar değer tek kenarın ağırlığı değil, kaynaktan o düğüme kadar olan yolun toplam ağırlığıdır.
- En kısa yol problemi gezgin satıcı problemiyle (traveling salesman) karıştırılmamalı. Burada tüm düğümleri tek turda gezmiyoruz; kaynaktan her düğüme ayrı ayrı en kısa yolu buluyoruz.
- Ödev: komşuluk matrisi klavyeden (kullanıcıdan) okunacak. "Dosyadan okunacak" diye bir şart yok. Asistanların duyurularını takip edin.

Kısa Tekrar Notları

- MST: tüm düğümleri bağlayan, döngüsüz, toplam kenar ağırlığı en küçük alt graf.
- Jenerik MST: $A = \emptyset$ ile başla. A bir kapsayan ağaç olmadığı sürece güvenli bir (u, v) kenarı bul, $A = A \cup \{(u, v)\}$ yap.
- Açgözlü (greedy) yaklaşım: her adımda güvenli kenarlar arasından en ucuzunu seç.
- Kruskal:
  - Her düğüm ayrı bir küme (`MAKE-SET`).
  - Kenarları ağırlığa göre artan sırada sırala.
  - Her kenar için uçlar farklı kümelerdeyse kenarı ekle ve kümeleri birleştir (`UNION`).
  - Karmaşıklık: $O(E \log V)$. Kenarlar sıralı gelirse neredeyse doğrusal.
- Prim:
  - Bir r kökünden başla, `key[r] = 0`, diğer tüm düğümler için `key = ∞`, `π = NIL`.
  - Kuyruktan `EXTRACT-MIN` ile en hafif kenarla bağlanan düğümü çek.

  - Komşularının anahtarını gerekirse azalt (`DECREASE-KEY`) ve ebeveynini güncelle.
  - İkili heap ile $O(E \log V)$. Fibonacci heap ile $O(E + V \log V)$.
- Dijkstra:
  - Başlangıç: `d[s] = 0`, diğerleri ∞, ebeveynler `NIL`.
  - Her adımda kuyruktan en küçük `d` değerli düğümü çek ve komşularını gevşet.
  - Gevşetme: $d[v] > d[u] + w(u, v)$ ise $d[v] = d[u] + w(u, v)$ ve $\pi[v] = u$.
  - Dizi ile $O(V^2)$. İkili heap ile $O(E \log V)$. Fibonacci heap ile $O(E + V \log V)$.
- Yol, ebeveyn (π) dizisinde hedeften kaynağa geri gidilerek bulunur.
- Tüm çiftler arası en kısa yol (all pairs shortest path) ayrı bir problemdir: örneğin Floyd algoritması.

Detaylı Açıklamalar

Jenerik MST ve Güvenli Kenar

- Köydeki her ev bir düğüm, her yol bir kenardır. Yol yapım maliyeti kenar ağırlığıdır ($w(u, v)$).
- Amaç: tüm evleri en az toplam maliyetle birbirine bağlamak.
- A kümesi her zaman sonuçta oluşacak bir MST'nin alt kümesi olmalıdır. MST'de olmayan bir kenar A'ya girerse MST elde edilemez.
- Kesim (cut): düğümlerin S ve $V - S$ diye iki parçaya ayrılması.
- İki parçayı bağlayan kenarlar arasında en hafif olanı seçmek, toplamı en küçük tutar ve ağaç yapısını bozmaz.
- Örnek: S ile $V - S$ arasında (x, y) ve (u, v) kenarları olsun. $w(u, v) < w(x, y)$ ise (u, v) seçilir.

Kruskal Algoritması

```
MST-KRUSKAL(G, w)
  A = ∅
  her v ∈ G.V için MAKE-SET(v)
  G.E kenarlarını w'ye göre artan sırada sırala
  her (u, v) kenarı için (sıralı)
    eğer FIND-SET(u) ≠ FIND-SET(v)
      A = A ∪ {(u, v)}
      UNION(u, v)
  return A
```

- Kitaptaki 9 düğümlü örneğin kenarları:

Kenar	Ağırlık	Kenar	Ağırlık
h-g	1	c-d	7
c-i	2	h-i	7
f-g	2	a-h	8
a-b	4	b-c	8
c-f	4	d-e	9
g-i	6	e-f	10
b-h	11	d-f	14

- Adımlar (eşit ağırlıklarda alfabetik sıra):

Adım	Kenar	Ağırlık	Karar
1	h-g	1	Eklendi
2	c-i	2	Eklendi
3	f-g	2	Eklendi
4	a-b	4	Eklendi
5	c-f	4	Eklendi: {c, i} ile {f, g, h} birleşti
6	g-i	6	Atlandı: aynı bileşen

Adım	Kenar	Ağırlık	Karar
7	c-d	7	Eklendi
8	h-i	7	Atlandı: aynı bileşen
9	a-h	8	Eklendi: {a, b} ana bileşene bağlandı
10	b-c	8	Atlandı: aynı bileşen
11	d-e	9	Eklendi: tek bileşen kaldı

- 11. adımdan sonra $V-1 = 8$ kenar seçildi. Kalan kenarlara bakmaya gerek yok. Bakılırsa hepsi aynı bileşende olduğu için atlanır.
- Toplam ağırlık: $1 + 2 + 2 + 4 + 4 + 7 + 8 + 9 = 37$.
- Hoca eşit kenarları ters sırada seçerek ikinci bir MST çizdi: `a-h` yerine `b-c` seçildi. Toplam yine 37. İki ağaç farklı, ikisi de doğru cevap.
- Karmaşıklık:
  - `MAKE-SET`: $O(V)$.
  - Kenarları sıralama: $O(E \log E) = O(E \log V)$.
  - Kenar döngüsü: $O(E)$ kez küme kontrolü.
  - Toplam: $O(E \log V)$.

Prim Algoritması

- Her adımda ağaçtaki düğümlerin (V_A) komşuları arasından, ağaçta olmayan ($V - V_A$) en hafif kenarla bağlanan düğüm eklenir.
- Komşuluk listesini her seferinde doğrusal taramak pahalıdır. Bu yüzden öncelikli kuyruk (min heap) kullanılır.
- Kuyrukta henüz ağaca eklenmemiş düğümler durur. Anahtar değer, düğümü ağaca bağlayan en hafif kenarın ağırlığıdır.
- Ağaca komşu olmayan düğümün anahtarı ∞'dur.

```
MST-PRIM(G, w, r)
  her u ∈ G.V için
    u.key = ∞
    u.π = NIL
  r.key = 0
  Q = G.V
  Q ≠ ∅ olduğu sürece
    u = EXTRACT-MIN(Q)
    her v ∈ G.Adj[u] için
      eğer v ∈ Q ve w(u, v) < v.key
        v.π = u
        v.key = w(u, v)      // DECREASE-KEY
```

- Aynı köy örneği, kök `a`:

Adım	Çekilen düğüm	Bağlandığı kenar	Ağırlık
1	a	(kök)	0
2	b	a-b	4
3	c	b-c	8
4	i	c-i	2
5	f	c-f	4
6	g	f-g	2
7	h	g-h	1
8	d	c-d	7
9	e	d-e	9

- 3. adımda `a-h` (8) ile `b-c` (8) eşittir. Hangisinin seçileceği gerçekleştirmeye bağlıdır.
- `i` eklenince `h`'nin anahtarı 8'den 7'ye düşer. `g` eklenince 1'e düşer.
- `d` eklenince `e`'nin anahtarı 10'dan 9'a düşer, ebeveyni `d` olur.
- Toplam ağırlık yine 37.

- Ebeveyn dizisi: `b.π = a`, `c.π = b`, `i.π = c`, `f.π = c`, `g.π = f`, `h.π = g`, `d.π = c`, `e.π = d`.
- Graf bağlı değilse veya yönlüyse bazı düğümlere ulaşılamayabilir. Bunlar ağaçta yer almaz.
- Karmaşıklık (ikili heap):
  - Kuyruğu kurma: $O(V \log V)$ (doğrusal kurulumla $O(V)$).
  - V kez `EXTRACT-MIN`: $O(V \log V)$.
  - En fazla E kez `DECREASE-KEY`: $O(E \log V)$.
  - Toplam: $O(E \log V)$.
- Fibonacci heap ile `DECREASE-KEY` ortalama (amortized) $O(1)$ olur. Toplam $O(E + V \log V)$. Ayrıntısı Algoritma Analizi dersinde.

En Kısa Yol Problemi

- Girdi: kenar ağırlıklı bir $G = (V, E)$ grafı ve bir s kaynak düğümü. Graf yönlü ya da yönsüz olabilir.
- Amaç: s'den ulaşılabilen her düğüme en kısa yolu bulmak.
- Ağırlık probleme göre değişir: kilometre, dakika vb.
- Kullanım alanları:
  - Ulaşım planlama ve harita uygulamaları (örneğin Yıldız kampüsünden Davutpaşa kampüsüne en kısa yol).
  - Ağlarda paket yönlendirme.
  - Sosyal ağ analizi, robotik, derleyiciler, zamanlama ve kaynak atama problemleri.
- Kaba kuvvetle tüm yolları çıkarıp karşılaştırmak pratikte imkânsızdır.
- Dijkstra algoritmayı 1956'da tasarladı, 1959'da yayımladı. Sözü: "This was the first graph problem I ever posed myself and solved."

Dijkstra Algoritması

```
DIJKSTRA(G, w, s)
  INITIALIZE-SINGLE-SOURCE(G, s)   // d[v] = ∞, π[v] = NIL, d[s] = 0
  S = ∅
  Q = G.V
  Q ≠ ∅ olduğu sürece
    u = EXTRACT-MIN(Q)
    S = S ∪ {u}
    her v ∈ G.Adj[u] için
      RELAX(u, v, w)

RELAX(u, v, w)
  eğer d[v] > d[u] + w(u, v)
    d[v] = d[u] + w(u, v)
    π[v] = u
```

Örnek 1: Elle çizilmiş yönlü graf (kaynak A)

- Kenarlar: A→B 4, A→C 2, C→B 1, C→D 4, C→E 5, B→C 3, B→D 2, B→E 3, E→D 1.

Adım	Çekilen	d[B]	d[C]	d[D]	d[E]
Başlangıç	A (0)	∞	∞	∞	∞
1	A	4 (A)	2 (A)	∞	∞
2	C (2)	3 (C)	2	6 (C)	7 (C)
3	B (3)	3	2	5 (B)	6 (B)
4	D (5)	3	2	5	6
5	E (6)	3	2	5	6

- 3. adımda `B→C` ile $3 + 3 = 6 > 2$ olduğu için C güncellenmez.
- 5. adımda `E→D` ile $6 + 1 = 7 > 5$ olduğu için D güncellenmez.
- Sonuç: C = 2, B = 3 (A→C→B), D = 5 (A→C→B→D), E = 6 (A→C→B→E).

Örnek 2: Kitaptaki yönlü graf (kaynak s)

- Kenarlar: s→t 10, s→y 5, t→y 2, t→x 1, y→t 3, y→x 9, y→z 2, x→z 4, z→x 6, z→s 7.
- Çekilme sırası: s (0), y (5), z (7), t (8), x (9).
- Önemli güncellemeler:
  - y çekilince t: 10 → 8, x: ∞ → 14, z: ∞ → 7.
  - z çekilince x: 14 → 13.
  - t çekilince x: 13 → 9.
- Sonuç: y = 5, z = 7, t = 8 (s→y→t), x = 9 (s→y→t→x).

Örnek 3: Yönsüz graf (kaynak A, hedef Z)

- Kenarlar: A-B 4, A-C 2, B-C 1, B-D 5, C-D 8, C-E 10, D-E 2, D-Z 6, E-Z 5.

Çekilen	Güncellemeler
A (0)	B = 4 (A), C = 2 (A)
C (2)	B = 3 (C), D = 10 (C), E = 12 (C)
B (3)	D = 8 (B)
D (8)	E = 10 (D), Z = 14 (D)
E (10)	Z için $10 + 5 = 15 > 14$, değişmez
Z (14)	Bitti

- A'dan Z'ye en kısa yol: A → C → B → D → Z, uzunluk 14.
- Yol, Z'den başlayıp ebeveynlere geri gidilerek bulunur: Z ← D ← B ← C ← A.

Ders sonrası notlar

- **Prim sözde kodu:** Hoca ilklendirmede "bütün kenarların anahtar değerleri sonsuz" ve "her U kenarı için π" dedi. Doğrusu düğümlerdir: her düğümün `key` değeri ∞, `π` değeri `NIL` yapılır.
- **Prim örneğindeki ebeveyn listesi:** Hoca bir yerde "B'nin parent'ı C" dedi. Doğru ebeveynler yukarıdaki tablodadır: `f.π = c`.
- **Dijkstra örnek 3:** Hoca B'den D'ye geçerken "B'ye gidiş maliyetim 8'di" dedi. B'nin uzaklığı 3'tür; $3 + 5 = 8$ D'nin yeni uzaklığıdır.
- **Negatif ağırlık:** Hoca kenarların "pozitif" olması gerektiğini söyledi, sonra $w(u, v) \geq 0$ yazdı. Doğru koşul "negatif olmayan" ağırlıktır; 0 ağırlıklı kenar sorun değildir.
- **Birden fazla MST:** Tüm kenar ağırlıkları farklıysa MST tektir. Birden fazla MST ancak eşit ağırlıklı kenarlar olduğunda ortaya çıkabilir.
- **Prim'de başlangıç:** Kuyruğu doğrusal sürede heap'e çevirmek (`BUILD-HEAP`) mümkündür. Bu toplam karmaşıklığı değiştirmez; baskın terim $O(E \log V)$'dir.

Duyurular

- Ödevde komşuluk matrisi kullanıcıdan (klavyeden) okunacak. Dosyadan okuma şartı yok. Değişiklik olursa asistanlar duyuracak.
- Hoca, iletişimin üniversitenin resmî kanalı (Online Yıldız) üzerinden yürümesini istiyor; ayrı bir Classroom açılmasını doğru bulmuyor.
- Vize kodlarının okunması seçim görevleri nedeniyle gecikti. Kalan 10-15 kod 2-3 gün içinde okunacak.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.